Machine Learning Pipelines and Erroneous Data

Lia Petrova, Mansi Saxena, Sai Chintapalli, Rahil S

ABSTRACT

Data errors are common in real-world datasets, but can substantially affect the performance of machine learning pipelines. This project investigates how different data corruption techniques affect machine learning pipelines and evaluates the effectiveness of various data cleaning strategies across textual, numerical, and categorical data. Our evaluation is conducted using two datasets: an Amazon product reviews dataset [8] for text-based prediction and an Adult Income dataset [3] for numerical and categorical data. We compare baseline performance of our pipelines against corrupted and cleaned data using traditional and manual cleaning methods, as well as automated frameworks, AutoClean [4] and Cleanlab [5]. The results show that both the performance degradation after corruption and the effectiveness of cleaning methods strongly depend on the data type and corruption mechanism and that novel methods are often better than simple and automatic cleaning.

1 INTRODUCTION

Data cleaning is an ever-present topic in data science, data processing, machine learning and all related fields that rely on data to produce results. Many surveys have testified that a large fraction of data engineers' time is spent namely on data validation, cleaning, standardisation and similar tedious but crucial preprocessing steps [1]. At the same time, the bigger and more complicated datasets are, the easier it is to miss errors in the data (e.g. impossible values, typos, bad formatting), especially if they are transferred in loose formats like JSON, CSV, or TXT. Errors in the data can occur for many different reasons, from human mistakes at the time of manual recording/labeling, through machine failure resulting in duplicated/dropped/corrupted entries, to malicious malformation of (most frequently sensitive and important) data.

In this work we research how data corruption techniques impact downstream performance, and which cleaning strategies are capable of mitigating them. We chose two datasets for supervised learning to demonstrate the effects of corruption and cleaning on different types of data. The first dataset consists of real Amazon product reviews and the text review field is used to predict the rating assigned to a product on a five-point scale (1–5). This task is a supervised text classification problem which uses text processing and sentiment analysis techniques. The corruption mechanisms included standard text-specific corruptions as well as basic structural injections using the JENGA framework [6], such as row dropping, duplication, and swapping. Natural language processing pipelines are sensitive to such perturbations. As a result, careful standardisation and cleaning are often required before training and evaluating machine learning models on textual data.

The second dataset is quite different in its structure and was used to demonstrate how our chosen models deal with numerical and categorical corruptions. It is a popular dataset for binary classification predicting the annual income (<= or >50K) of adults based on various numerical and categorical features, such as the age, workclass, education level, etc. Modern machine learning models are not as sensitive to injections in numerical and categorical data as in textual data, so a more variable ensemble of corruptions was performed with cleaning methods.

This report presents a summary of our work investigating the effects of data corruption and cleaning methods in machine learning pipelines. The Problem & Approach section describes the research questions we aimed to address, the approach used to tackle them, and provides detailed information about the implementation of our pipelines, models, corruption and cleaning methods, and the evaluation metrics used. The Evaluation section contains a detailed setup of our experiments, including infrastructure, data size, baselines, and figures that we believe contribute to an overall understanding of the problem. In the final section, Discussion and Conclusion, we summarise our main findings, reflect on limitations encountered during the project, discuss challenges faced, and outline potential directions for future work to extend our research.

Contributions.

- The first contribution is exploring how different data corruption methods affect machine learning model performance across commonly used data types - textual, numerical, and categorical -how effectively performance can be recovered after cleaning, and the resulting changes in data statistics under the different data manipulations.
- The second contribution focuses on investigating the effectiveness of different cleaning techniques on different types of data corruption. Standard cleaning methods appropriate to each data type are used alongside more recent frameworks, such as AutoClean and Cleanlab, to evaluate which approaches are most effective in mitigating data corruption.

2 PROBLEM & APPROACH

Problem Statement.

Different data types suffer from different corruptions, requiring diverse cleaning techniques. Since these corruption and cleaning methods cannot be directly compared in terms of their influence on models, we built a supervised learning pipeline for various data types (textual, numerical, and categorical) and studied the effect of errors on the pipeline's performance. We focused on identifying which corruption methods are the most harmful, how different cleaning strategies affect performance, and which cleaning methods are most effective for textual, numerical, and categorical data.

Approach.

This project followed two approaches, one for each dataset.

Amazon Reviews: We first trained a model on the original dataset, using the review text to predict the rating, to establish baseline accuracy and statistics. These baseline metrics, were used to evaluate textual corruption and cleaning effects. In each iteration, the model was trained and evaluated on corrupted data, then cleaned using the different techniques and re-evaluated. All accuracies and statistics were recorded to assess the effectiveness of each technique.

Adult Income: Similar to the Amazon dataset, we established a baseline, measured the impact of corruption, and evaluated performance recovery after cleaning. The primary difference is the evaluation granularity: due to the dataset's high feature count, corruptions were grouped into numerical and categorical types rather than individual methods. Corrupting only a small subset of features often did not significantly degrade performance. The most substantial performance degradation was seen when both numerical and categorical corruptions were applied simultaneously.

Implementation.

Amazon Reviews: sickit-learn's [7] logistic regression model with a dual TF-IDF (word and character) feature extraction was used. Nine corruption methods were applied with varying intensities: single-attribute corruptions included injecting missing text (30%), broken characters (25%), or swapped values (20%) into reviews, and missing (15%) or swapped (12%) rating labels. Multi-attribute methods combined these techniques, such as broken characters with missing text, swapping both text and labels, or applying more intense missing values to both fields. Finally, the last approach applied all 4 corruption types-broken characters, swapped text, missing text, and swapped labels-simultaneously at lower percentages.

Four cleaning strategies, each targeting different types of issues, were implemented and evaluated.

The first one was basic cleaning, performing minimal preprocessing for structural consistency. Rows with missing labels were removed, and missing/empty text fields were replaced with a placeholder token. Text was standardized by lowercasing, normalizing whitespace, and correcting encoding issues using the ftfy library. Labels were constrained to the valid rating range (1–5).

The second approach was heuristic cleaning, done to remove outliers/low-quality samples. In addition to dropping rows with missing labels, this method filtered reviews based on character length thresholds, alphabetic character ratios, vocabulary richness, excessive repeated characters, and patterns indicating URLs or garbled non-ASCII text.

The third approach was semantic cleaning, which tried to preserve the meaning of the text while improving quality. It expanded common abbreviations, reduced exaggerated character repetitions, and removed unwanted characters. Negations were handled explicitly by joining negation terms with the following token (e.g., "not good" to "not_good") to retain sentiment information. The cleaned text was lemmatized using NLTK's WordNetLemmatizer.

The final approach was a model-aware cleaning, using Cleanlab's CleanLearning framework. Rather than removing data points, this identified label issues by estimating label quality scores using a cross-validated logistic regression pipeline. These scores were used to assign sample weights during training, allowing the model to down-weight likely mislabeled examples instead of removing them.

Adult Income.: Since this contains both numerical and categorical features, we used a preprocessing pipeline consisting of standard scaling for numerical attributes and one-hot encoding for categorical ones. We evaluated two classifiers from the scikit-learn library [7]: Logistic Regression and the Histogram-based Gradient Boosting Classifier. These were chosen due to their differing complexity and handling of missing data. While logistic regression is a fast linear model, it cannot natively handle missing values, and required imputation during the experiments. In contrast, the Histogram Gradient Boosting Classifier supports missing values internally, allowing us to observe the effects of corruption without additional manipulation.

We applied corruption techniques affecting both numerical and categorical features. Basic corruptions, such as missing values, row duplication, and value swapping were done using the JENGA framework. Since JENGA provides limited numerical corruptions (Gaussian noise and scaling), we additionally implemented constraint violations, negation, and the injection of textual characters into numeric columns. Categorical corruptions included category shifts, typos, and missing values. Corruptions were applied to 25% of the data, as lower levels had minimal impact on performance and higher levels led to unstable models.

For cleaning, we first applied standard preprocessing steps, including removal of non-numeric characters, absolute value correction, and imputation of missing values, followed by capping scaled values. We then used AutoClean for handling outliers, duplicates, and missing values, including categorical attributes. Finally, we applied Cleanlab on top of the cleaned data to identify and remove rows more than 98% likely to harm model performance. This step removed fewer than 10% of the samples while improving classification accuracy by approximately 10-12%, representing a reasonable trade-off between discarded data and performance.

3 EVALUATION

Experimental Setup.

We evaluated cleaning effectiveness using a 4-part framework: standard ML metrics and data retention statistics, TensorFlow Data Validation, statistical analysis, and computational cost–benefit analysis. The Amazon dataset was evaluated using 9 corruption types applied individually. Each corruption was tested across 4 cleaning strategies, resulting in 45 total experiments. The Adult dataset evaluated combined numerical and categorical corruptions. Two models were used: Logistic Regression and HistGradientBoosting. Cleaning was applied in two stages: standard methods and model-aware cleaning using Cleanlab. Multiple runs per configuration resulted in 75 experiments. Statistical analysis measured recovery rates, metric consistency, distribution shifts using Kolmogorov–Smirnov tests, and cost–benefit trade-offs. Key libraries used include scikit-learn 1.3+ for model training, JENGA 1.0 for corruption injection, AutoClean 2.1 and Cleanlab 2.4 for cleaning, TensorFlow Data Validation 1.15 for schema validation, and SciPy 1.11 for statistical testing.

Datasets.

Amazon Reviews 2023 [8] We used the 'text' column from this dataset to predict the target 'rating' [1-5]. For a balanced distribution across the five rating classes, we sampled each class for 20,000 rows. This resulted in a balanced pool of 100,000 total records, which was used for training and testing.

Adult Income [3] The Adult Income dataset contains information on whether an individual's annual income exceeds $50,000, which was the prediction target in the 'income' column. We used numerical features - 'age', 'educational-num', 'capital-gain', 'capital-loss', and 'hours-per-week', and categorical features - 'workclass', 'marital-status', 'occupation', 'relationship', 'race', and 'gender' - to predict income. The dataset was resampled to include 10,000 instances per class to balance the labels and reduce bias.

Experiments and results.

Adult Income Dataset: The experiments examined how model accuracy changes when trained on original, corrupted, and cleaned data using two prediction models. As shown in Figure 1, the models behave differently. Logistic regression, a simple linear model, suffers greatly from data corruption, with accuracy dropping from nearly 80% to around 50%. Accuracy improves with basic cleaning and almost fully recovers when Cleanlab-identified labels are removed. In contrast, the histogram gradient boosting classifier experiences only a small accuracy drop with corrupted data but deteriorates more with standard cleaning. This likely reflects its better handling of NaNs, as its advanced methods outperform the simple injections and scaling used in cleaning. Accuracy improves only after applying Cleanlab, though it does not fully recover. More results can be found in the Appendix

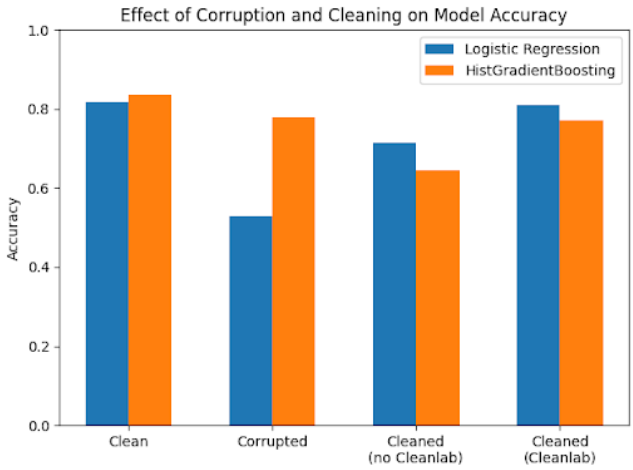

Figure 1: Overview of the accuracy changes at different stages of the data manipulation.

Amazon Reviews 2023 Dataset:

Data Corruption: Baseline accuracy was 51.52%. Missing text caused the largest drop to 41.77%, while combined text and label gaps reduced accuracy to 43.06%, highlighting the importance of text for sentiment analysis. Missing labels caused crashes due to the inability to train. "All Corruptions" reduced accuracy to 46.13%, indicating that noise effects do not combine linearly. **Cleaning Strategies Effectiveness**: Model-Aware cleaning was most effective, averaging 49.30% accuracy and exceeding the baseline for broken characters and missing labels (53.29% and 53.93%). Heuristic cleaning followed (48.94%), performing well in high-missingness cases such as "Missing Text" (51.16%) due to aggressive row removal, which introduces bias. Semantic and Basic cleaning were less effective, with Semantic sometimes underperforming corrupted data (e.g., -3.4% in "All Corruptions") due to sentiment-altering replacements. **Data Trade-offs**: Heuristic cleaning dropped substantial data (11.75% on average;

up to 33.7% in Heavy Missing). Semantic cleaning modified rows, while Model-Aware methods retained nearly all data via reweighting, achieving high accuracy with minimal data loss. **Computational Analysis**: Basic and Heuristic methods were fast and scalable, while Semantic and Model-Aware cleaning incurred higher latency from text processing. Heuristic reduced evaluation time by shrinking the dataset, but overall gains from traditional methods were marginal relative to cost, making Model-Aware cleaning preferable despite added overhead.

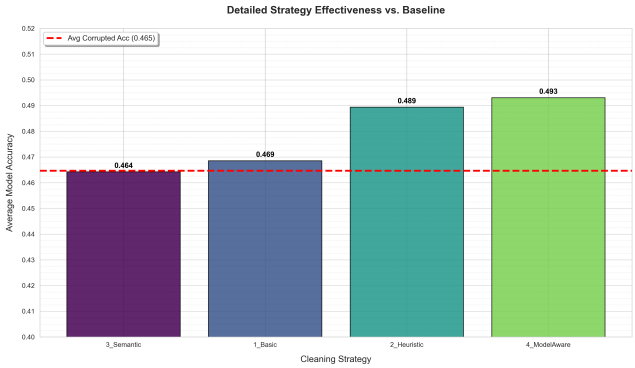

Figure 2: Average model accuracy for different cleaning methods.

Statistical Testing and TFDV - Both Datasets.
Statistical tests confirm that cleaning gains are causal. On the Amazon dataset, Model-Aware cleaning improved accuracy in 7 of 9 corruption types, with mean gains of 2.8% and improvements across 2.8 of 3 metrics on average. KS tests detected significant text-length shifts for missing-value corruptions ($p < 0.001$). On the Adult Income dataset, 75 experiments across two models showed divergent effects: Logistic Regression recovered accuracy across all four metrics, while HistGradientBoosting degraded after Cleanlab cleaning. KS tests found significant shifts ($p < 0.001$) in all numeric features. Overall, model-aware cleaning benefits linear models but can harm tree-based models by removing split-critical data.

Google's production grade validation library, TFDV [2] detects data issues via schema validation and drift analysis by comparing corrupted data to a baseline schema. Detection varied by dataset: Adult data achieved 100% detection with 20.9 anomalies per corruption, while Amazon data reached 22% with 0.67 anomalies. Structural corruptions were easily detected, but semantic corruptions (e.g., label swaps) caused minimal schema violations despite accuracy drops, showing schema validation is insufficient for text data.

Question	Test	Adult Dataset	Amazon Dataset
Does cleaning improve?	Recovery	LogReg: 95%; HistGB: -107%	Model-aware wins 7/9; +2.8% avg
Which method best?	Level comp.	LogReg: Cleanlab; HistGB: Baseline	Model-aware: 7/9; Heuristic: 2/9
Are gains meaningful?	Cost-benefit	LogReg: 0.059; HistGB: 0.035	+1.9–3.7%, corruption-dep.
Gains consistent?	Multi-metric	LogReg: 4/4; HistGB: 0/4	9/9 improve; 2.8/3 metrics
Distribution shifts?	KS test	All 4: $p < 0.001$	Text length shifts

Table 1: Statistical validation of cleaning effectiveness across datasets.

4 DISCUSSION AND CONCLUSION

Through our project we were able to test the effects of different types of corruptions and determine the effectiveness of our cleaning techniques on textual, numerical, and categorical data. We concluded that different data types and models require tailored pipelines often based on the domain, but automated and model-aware cleaning methods, such as Cleanlab, can positively influence the accuracy of different models.

Due to the time constraint, we limited the scope of the project to standard and automatic leaning techniques, and data corruptions that could happen in real life. Another expected limitation was how difficult it would be to differentiate between anomalous and corrupted data, specifically in the textual data - natural language can be unpredictable and sometimes injected corruptions cannot be differentiated from anomalous data.

Future extensions of the project would include exploring other types of data corruption, such as image or audio, and appropriate cleaning techniques. We could also perform tests on more datasets and datatypes, in order to see if there are other variables that impact the effect of data corruption and cleaning techniques.

Code. The code for this project is publicly available at https://github.com/mansivsaxena/data-errors-ml-pipelines/tree/main.

Detailed Contributions.

- Sai Chintapalli: Developed data corruption functions used in the pipelines. Refactored adult income evaluation code.
- Lia Petrova: Developed the adult income dataset pipeline (ML models, data processing, and cleaning functions). Worked on the Introduction and Adult Income sections of the presentation and the report.
- Mansi S: Developed the Amazon Reviews data pipeline (model, cleaning, evaluation experimental set-up) and visual analysis (data, performance, computation, Cleanlab - graphs and plots). Structured repository directories and covered respective portions of the presentation and report.

- Rahil S: Designed 4-part evaluation framework and implemented statistical testing pipelines with TFDV integration for both datasets. Wrote Evaluation section and created repository documentation.

REFERENCES

[1] Anaconda Inc. 2022. State of Data Science Report 2022. https://www.anaconda.com/resources/whitepapers/state-of-data-science-report-2022/. Accessed: 2026-01.

[2] Denis Baylor, Eric Breck, Heng-Tze Cheng, et al. 2017. TFX: A TensorFlow-Based Production-Scale Machine Learning Platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1387–1395. https://doi.org/10.1145/3097983.3098021

[3] Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5XW20.

[4] Elise Mercury. 2021. AutoClean: Automated Data Preprocessing and Cleaning. https://github.com/elisemercury/AutoClean/blob/main/README.md. GitHub repository.

[5] Curtis Northcutt, Jonas Mueller, and Anish Athalye. 2022. cleanlab 2.0: Automatically Find Errors in ML Datasets. https://cleanlab.ai/blog/learn/cleanlab-2/. Accessed: 2026-01.

[6] Nikolaos Papadopoulos, Panagiotis Papapetrou, Alexandros Nanopoulos, and Yannis Theodoridis. 2021. JENGA: A Framework to Study the Impact of Data Errors on the Predictions of Machine Learning Models. In *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. 134–145. https://openproceedings.org/2021/conf/edbt/p134.pdf

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, and Édouard Duchesnay. 2011. *scikit-learn: Machine Learning in Python*. scikit-learn developers. Documentation available at https://scikit-learn.org/stable/.

[8] Zhankui He An Yan Xiusi Chen Julian McAuley Yupeng Hou, Jiacheng Li. [n.d.]. Bridging Language and Items for Retrieval and Recommendation. https://cseweb.ucsd.edu/~jmcauley/datasets.html#amazon_reviews Accessed: 2026-01.

A ADULT INCOME DATASET STATISTICS

Due to space limitations only the most important statistics were presented in the Results section, but here are 2 more results of our Experiments. Figure 3 shows the differences in time taken for training and evaluation on a Google Colab notebook for both models. The 'Baseline' column addresses the time for training on the original data and naturally takes the shortest. The 'Corrupted' column shows that longer time was taken for training on corrupted data as now the logistic regression model has a harder time learning the features. The 'Cleaned' cleaning section and the 'Cleanlab' cleaning section show the time without and with Cleanlab, respectively.

Another statistics worth to consider is the following confusion matrix for the logistic regression classifier (figure 4) ran on the test set, which can help us observe the errors the model made. The most interesting is the difference in the corrupted classes which predominantly predicted one of

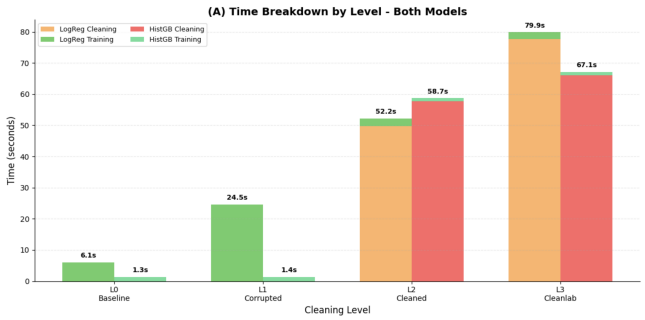

Figure 3: Time spent for training and cleaning each model.

the classes, resulting in around 50% of the accuracy for the balanced dataset.

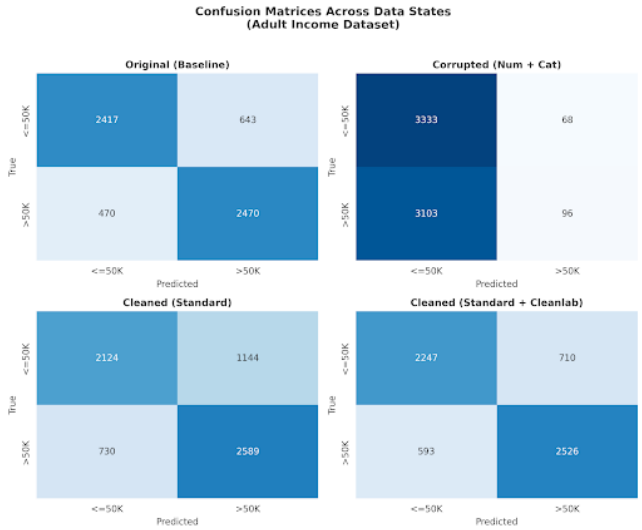

Figure 4: Confusion matrix of the logistic regression model

B AMAZON REVIEWS 2023 DATASET PLOTS

Due to space limitations, the comprehensive results and ablation analysis figures of 9 corruptions x (4+1) cleaning methods could not be attached in the main report. Figure 5 details accuracies obtained on applying each cleaning strategy to each corruption type. This helps support our observations and justifications listed in the results section as to which method works best for which corruption type.

For a cost-benefit analysis, we plot the accuracies against the total time taken to clean and evaluate the model, the analysis of which has been covered in the results section above. Figure 6 is a visual representation of the same.

Figure 5: Strategy-wise Accuracies per Corruption Method

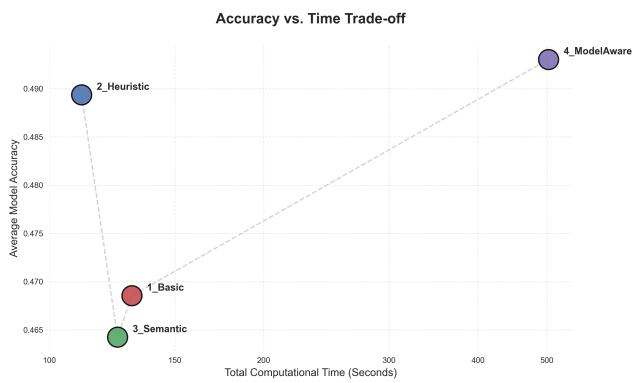

Figure 6: Accuracy vs Computational Time Trade-off

For a more detailed look into the Model-Aware cleaning method which uses Cleanlab's functions, we plotted the class noises detected per corruption type by Cleanlab in a heatmap. Figure 7 shows that rating classes 1 and 5 tend to be more stable and less noisy, whereas 2, 3, and 4 entail more noise. This can be justified by understanding the fact that human entered rating generally tend to pivot towards the extremes and classes like 2, 3 are generally more ambiguous because of less correlation with textual reviews.

These and all further graphs, figures, results and analysis files are available in the GitHub repository linked above.

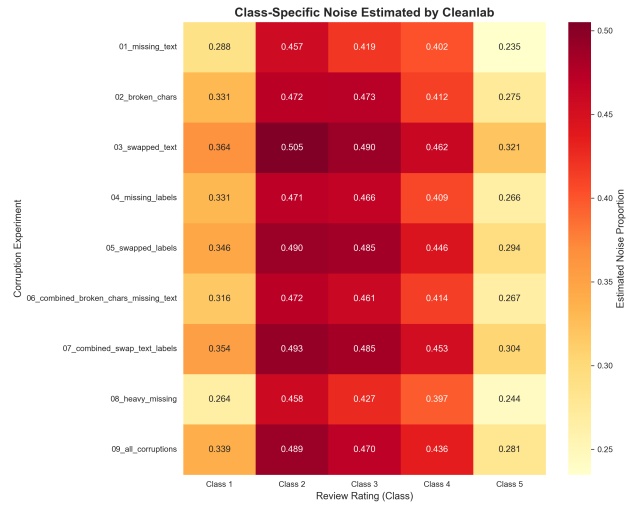

Figure 7: Class-wise Noise Heatmap - Generated by Cleanlab's CleanLearning model